Module I

Review of neural networks: Model of a biological neuron, McCulloch Pitts neuron, Activation functions, perceptron, perceptron learning algorithm & convergence, multilayer perceptron, back propogation, learning XOR, sigmoid neurons, Gradient descent, feed forward neural networks.

Module II

Training neural networks: Initialisation, dropout, batch normalisation & dropout, overfitting, underfitting, early stopping & validation curves.

Data visualisation: feature & weight visualisation, tSNE.

Introduction to Tensorflow: graph, nodes, Tensor data structures - rank, shape, type, building neural networks with tensorflow, Introduction to Keras.

Module III

Convolutional neural networks: convolution operation, convolutional layers in neural network, pooling, fully connected layers.

Case study: Architecture of Lenet, Alexnet & VGG 16

Module IV

Recurrent neural networks: Back propogation, vanishing gradients, exploding gradients, truncated backpropagation through time, Gated Recurrent units,

Long short-term memory (LSTM) cells, Solving vanishing gradient problems with LSTMs.

Module V

Autoencoders, variational autoencoders.

Generative adversarial networks (GAN): discriminative & generative models, GAN discriminator, GAN generator, upsampling, GAN training, GAN challenges, loss functions, cross entropy minimax loss, wasserstein loss.

→ AI is the way of making computer / robot / software to think like human being.

→ John Maccarthy is the father of AI.

→ AI's goal is to implement human intelligence in computer.

Machine learning

- ML is the subset of AI
- father of ML is Arthur Samuel
- It provides the s/m to automatically learn & improve from experiences & predict - output without being explicitly programmed.

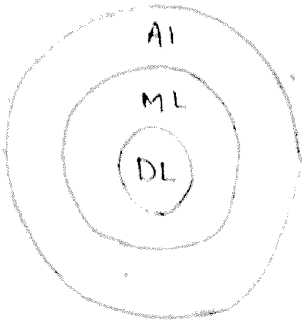
Eg: face recognition, cyber fraud detection, self driving cars, friend suggestion in facebook.

- It is a structured data representation.
- in ML, thousands of data points are used
- o/p is in the form of numerical data.

Deep learning

* DL is the subset of ML
* Jeffery Hinton is the father of DL
* DL uses structured model & is completely based on neurons.
* It can be called artificial Neural Network (ANN)
* ANN is used for data representation.
* Millions of data points are used.

* DL is used to solve Complex programmes
* DL o/p can be numerical, audio o/p, text o/p etc



Learning

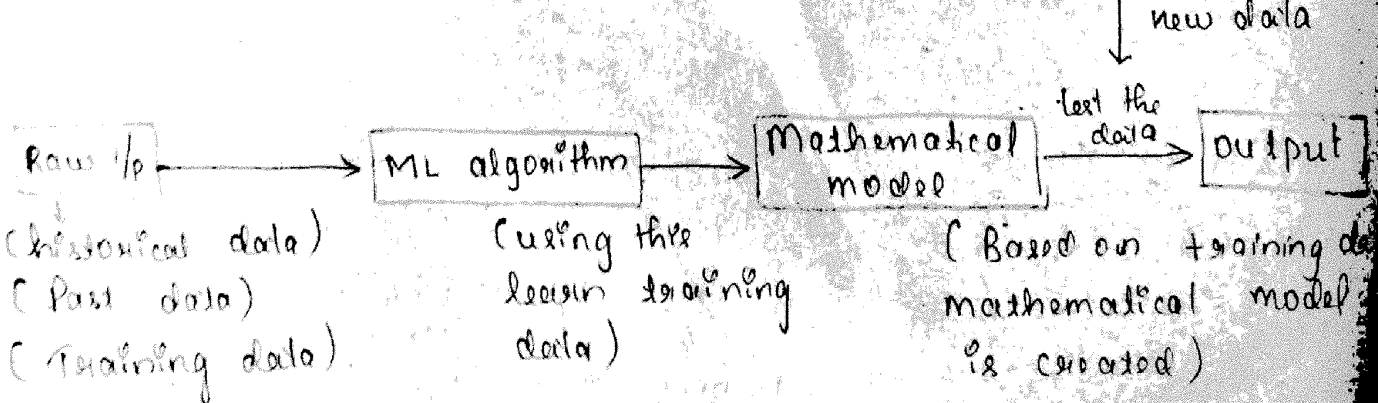
SUPERVISED LEARNING | UNSUPERVISED LEARNING | REINFORCEMENT LEARNING

In Supervised learning, machine uses labelled data. using labelled data, we are representing o/p in classification or regression

In Unsupervised learning, machine uses unlabelled data. Here grouping or clustering based on some similar patterns. It is called clustering or association.

Reinforcement learning is a feedback type, i.e, machine learn by doing action. Good actions get reward & bad actions get penalty.

Good Action ⟶ Reward
Bad Action ⟶ Penalty



Neural Network

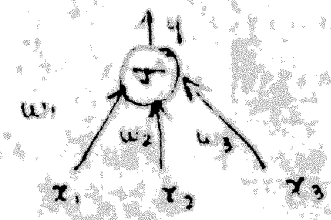
- one set of algorithm
- designed to recognise a pattern
- modelled on human brain
- It can take raw input
  - can be labelled or unlabelled
  - clustering or grouping based on similar patterns

o/p is two the form of numerical data in vector format for which we can translate all the real world problems

- It is used for feature extraction to fed into other algorithm

Biological neuron

* fundamental unit of deep neural network is called artificial neurons



* Artificial neurons are designed on ~~base~~ of basis of biological neuron

* Biological neurons are cells

How biological neuron look like :-

1) DENDRILE → tree structure
   → receive signals from other neurons

2) SYNOPSE → point of connection to other neurons
   → connection neuron + giamd
   → communication b/w neurons of various

3) SOMA → part where processing is done

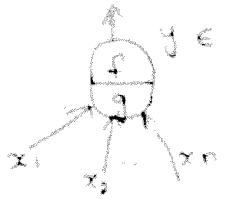
4) AXON → It transmits o/p to other parts of body

Eg: When a person hears a joke

Eg: Visualization of an image

McCulloch Pitts Model

- High computational model of neurons



- g is the aggregate function
- f is the function on which decision based on g and

made

- Inputs → Excitatory
  → Inhibitory

INHIBITORY

- Excitatory means preventing action to get exact o/p (y
- EXITATORY means more involved in producing o/p

$$g(x) = x_1 + x_2 + x_3 + \dots + x_n = \sum_{i=1}^{n} x_i \quad (y=1)$$

→ $y = f(g(x)) = 1$

$g(x)$ → Sum of input

when $g(x) \geq \theta$

where θ = threshold Parameter / threshold logic

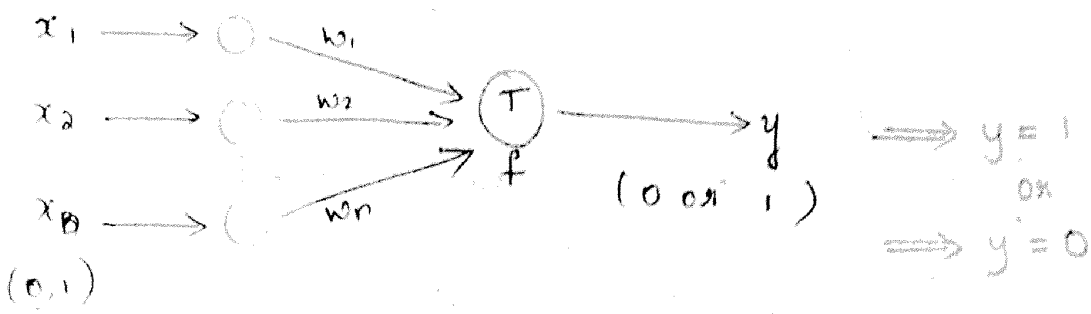
→ $y = f(g(x)) = 0$

when $g(x) < \theta$

→ 2 layers

• i/p layer (any no of i/p)

• o/p layer (only one o/p)



output $y=1$, $g(x)$ = sum of weighted i/p

$\Sigma x_i w_i \geq 0$

$y=0$, $\Sigma x_i w_i < 0$ → threshold value or threshold parameter or BIAS

↳ minimum value of weighted i/p from neurons to fire.

. weight (w) can take +1, -1

-1 → Inhibitatory (Prevents the action)

+1 → Excitatory (Support the action)

Q. $x_1 = 1$ $x_2 = 1$ $x_3 = 0$

$w_1 = +1$ $w_2 = +1$ $w_3 = -1$

* → $x_1 w_1 = 1 \times +1 = +1$ $x_2 w_2 = 1 \times +1 = +1$ $x_3 w_3 = 0 \times -1 = 0$//

→ $\Sigma x_i w_i = x_1 w_1 + x_2 w_2 + x_3 w_3$

→ $\Sigma x_i w_i = 1 + 1 + 0 = 2$//

* Set threshold to set $y = 1$

Suppose $T = 1$, $\Sigma x_i w_i \geq T$

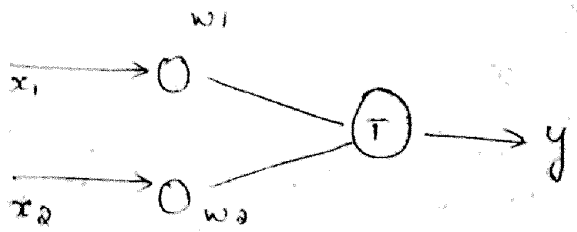
$2 \geq T$

$z = 1$

Threshold Satisfied $\Rightarrow$ $\boxed{y = 1}$

Implementation of OR Gate

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1



Suppose $w_1 = +1$ $w_2 = +1$ $T = +1$

$(0,0) \rightarrow \sum x_i w_i = 0 \times +1 + 0 \times +1 = 0$ // $0 \geq 1$ ✗ $y = 0$

$(0,1) \rightarrow \sum x_i w_i = 0 \times +1 + 0 \times +1 = +1$ // $1 \geq 1$ ✓ $y = 1$

$(1,0) \rightarrow \sum x_i w_i = 1 \times +1 + 0 \times +1 = +1$ // $1 \geq 1$ ✓ $y = 1$

$(1,1) \rightarrow \sum x_i w_i = 1 \times +1 + 1 \times +1 = +2$ // $2 \geq 1$ ✓ $y = 1$

For OR Gate: $w_1 = +1$ $w_2 = +1$

$T = +1$

Implementation of XOR Gate

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

① $w_1 = +1$ $w_2 = +1$

② $w_1 = +1$ $w_2 = -1$

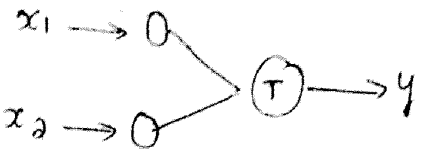
③ $w_1 = -1$ $w_2 = +1$

④ $w_1 = -1$ $w_2 = -1$

XOR can be written as $x_1 \cdot \bar{x}_2 + \bar{x}_1 \cdot x_2$

(1) $x_1 \cdot \bar{x}_2$ =

x_1	x_2	y
0	0	0
0	1	0
1	0	1
1	1	0



(i) Suppose $w_1 = +1$ $w_2 = +1$ & $T = 1$

(0,0) $= \Sigma x_i w_i = 0\times1 + 0\times1 = 0$ $0 \geq 1$ X $y = 0$

(0,1) $= \Sigma x_i w_i = 0\times0 + 1\times1 = 1$ $1 \geq 1$ ✓ $y = 1$

(1,0) $= \Sigma x_i w_i = 1\times1 + 0\times1 = 1$ $1 \geq 1$ ✓ $y = 1$

(1,1) $= \Sigma x_i w_i = 1\times1 + 1\times1 = 2$ $2 \geq 1$ ✓ $y = 1$

Not satisfied with the original T.T.

(2) Suppose $w_1 = +1$ $w_2 = -1$ & $T = 1$

(0,0) $= \Sigma x_i w_i = 0\times1 + 0\times-1 = 0$ $0 \geq 1$ X $y = 0$

(0,1) $= \Sigma x_i w_i = 0\times1 + 1\times-1 = -1$ $-1 \geq 1$ X $y = 0$

(1,0) $= \Sigma x_i w_i = 1\times1 + 0\times-1 = 1$ $1 \geq 1$ ✓ $y = 1$

(1,1) $= \Sigma x_i w_i = 1\times1 + 1\times-1 = 0$ $0 \geq 1$ X $y = 0$

So conditions satisfied

for XOR gate : $w_1 = +1$ $w_2 = -1$

$(x_1 \cdot \bar{x}_2)$ $T = 1$

(2) $\bar{x}_1 \cdot x_2$ =

x_1	x_2	y
0	0	0
0	1	1
1	0	0
1	1	0

(3) Suppose $w_1 = -1$ & $w_2 = +1$ & $T = 1$

$(0,0) = \Sigma x_i w_i = 0 \times -1 + 0 \times +1 = 0$ // $\quad 0 \geq 1$ X $\quad (y = 0)$

$(0,1) = \Sigma x_i w_i = 0 \times -1 + 1 \times 1 = 1$ $\quad 1 \geq 1$ ✓ $\quad (y = 1)$

$(1,0) = \Sigma x_i w_i = 1 \times -1 + 0 \times 1 = -1$ $\quad -1 \geq 1$ X $\quad (y = 0)$

$(1,1) = \Sigma x_i w_i = 1 \times -1 + 1 \times 1 = 0$ $\quad 0 \geq 1$ X $\quad (y = 0)$

Conditions Satisfied

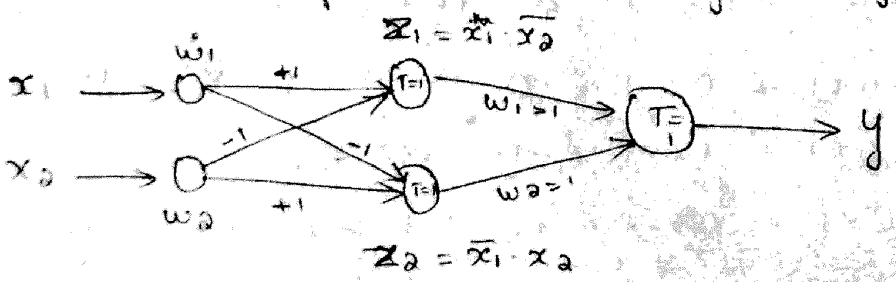
For XOR gate = $w_1 = -1$ $\quad w_2 = +1$

$x_1 \cdot x_2$ $\quad T = 1$

XOR =



We use extra one layer called <u>hiddon layer</u> because we cannot implement XOR only using the above layers.



XOR = $\underbrace{x_1 \cdot \bar{x}_2}_{z_1} + \underbrace{\bar{x}_1 \cdot x_2}_{z_2}$ $\quad (z_1 \underline{\text{OR}} \; z_2)$

↳ $w_1 = 1$

$w_2 = 1$

$T = 1$

(HW) Implementation of AND gate

	x_1	x_2	y
	0	0	0
	0	1	0
	1	0	0
	1	1	1

① Suppose $w_1 = +1$ $w_2 = +1$ $T = +1$

$(0,0) = \sum x_i w_i = 0\times1 + 0\times1 = 0$ $0 \geq 1$ ✗ $y=0$

$(0,1) = \sum x_i w_i = 0\times1 + 1\times1 = 1$ $1 \geq 1$ ✓ $y=1$

$(1,0) = \sum x_i w_i = 1\times1 + 0\times1 = 1$ $1 \geq 1$ ✓ $y=1$

$(1,1) = \sum x_i w_i = 1\times1 + 1\times1 = 2$ $2 \geq 1$ ✓ $y=1$

Condition not satisfied

② Suppose $w_1 = +1$ $w_2 = -1$ $T = +1$

$(0,0) = \sum x_i w_i = 0\times1 + 0\times-1 = 0$ $0 \geq 1$ ✗ $y=0$

$(0,1) = \sum x_i w_i = 0\times1 + 1\times-1 = -1$ $-1 \geq 1$ ✗ $y=0$

$(1,0) = \sum x_i w_i = 1\times1 + 0\times-1 = 1$ $1 \geq 1$ ✓ $y=1$

$(1,1) = \sum x_i w_i = 1\times1 + 1\times-1 = 0$ $0 \geq 1$ ✗ $y=0$

Condition not satisfied

③ Suppose $w_1 = -1$ $w_2 = +1$ $T = +1$

$(0,0) = \sum x_i w_i = 0\times-1 + 0\times1 = 0$ $0 \geq 1$ ✗ $y=0$

$(0,1) = \sum x_i w_i = 0\times-1 + 1\times1 = 1$ $1 \geq 1$ ✓ $y=1$

$(1,0) = \sum x_i w_i = 1\times-1 + 0\times1 = -1$ $-1 \geq 1$ ✗ $y=0$

$(1,1) = \sum x_i w_i = 1\times-1 + 1\times1 = 0$ $0 \geq 1$ ✗ $y=0$

Condition not satisfied

(4) Suppose $w_1 = -1$ $w_2 = -1$ $T = 1$

$(0,0) = \sum x_i w_i = 0 \times -1 + 0 \times -1 = 0$ $\quad 0 \geq 1$ ✗ $\quad y = 0$

$(0,1) = \sum x_i w_i = 0 \times -1 + 1 \times -1 = -1$ $\quad -1 \geq 1$ ✗ $\quad y = 0$

$(1,0) = \sum x_i w_i = 1 \times -1 + 0 \times -1 = -1$ $\quad -1 \geq 1$ ✗ $\quad y = 0$

$(1,1) = \sum x_i w_i = 1 \times -1 + 1 \times -1 = -2$ $\quad -2 \geq 1$ ✗ $\quad y = 0$

Condition not satisfied

(5) Suppose $w_1 = +1$ $w_2 = +1$ $T = 2$

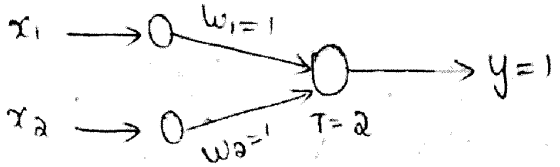
$(0,0) = \sum x_i w_i = 0 \times 1 + 0 \times 1 = 0$ $\quad 0 \geq 2$ ✗ $\quad y = 0$

$(0,1) = \sum x_i w_i = 0 \times 1 + 1 \times 1 = 1$ $\quad 1 \geq 2$ ✗ $\quad y = 0$

$(1,0) = \sum x_i w_i = 1 \times 1 + 0 \times 1 = 1$ $\quad 1 \geq 2$ ✗ $\quad y = 0$

$(1,1) = \sum x_i w_i = 1 \times 1 + 1 \times 1 = 2$ $\quad 2 \geq 2$ ✓ $\quad y = 1$

Condition satisfied.



For AND condition, ~~$x_1 = 1$~~ $w_1 = 1$ $w_2 = 1$ & $T = 2$

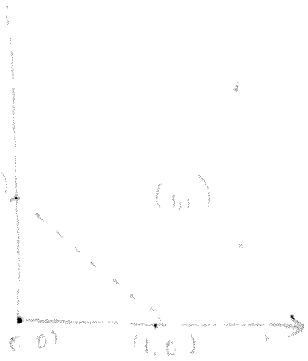
1/12/2021

Geometrical Representation of OR

$y = 1$ if $\sum_{i=1}^{n} x_i \geq \theta$

$y = 0$ if $\sum_{i=1}^{n} x_i < \theta$

0	0	0
0	1	1
1	0	1
1	1	1



Binary data are splitted into 4 portions such that inputs producing $y=1$ will be on one side & $y=0$ will be on other side.

Single neuron can be used to represent boolean functions which is linearly separable (using MP)

For 2 i/p ⟶ there exist a line
For 3 i/p ⟶ there exist a plane
↓
Similar to line.
$y=0$ on one side &
$y=1$ on other side

For non-boolean,

Classical Perception Model

Perceptron ⟶ Simple model based on biological model used in ANN

This model was refined & analysed by Minsky (1969) & Papert (1969). This model was referred as PERCEPTRON MODEL.

- Using this model, we can represent
  - → non-boolean
  - → non-linearly separable
  - → real data
  - → focus on weightage

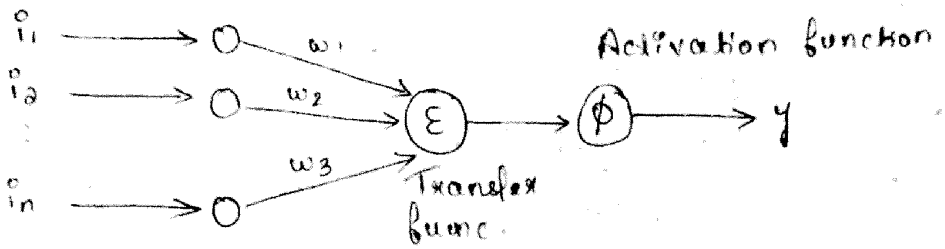
Here $y = 1$ if $\sum_{i=1}^{n} x_i w_i \geq \theta$ → Bias

$y = 0$ if $\sum_{i=1}^{n} x_i w_i < \theta$

or

$y = 1$ if $\sum_{i=1}^{n} x_i w_i - \theta \geq 0$

$y = 0$ if $\sum_{i=1}^{n} x_i w_i - \theta < 0$



Activation function : It is a func. used in ANN which outputs a small value when i/p are small. & it produces a larger value as o/p when i/ps exceeds threshold value (used to produce a o/p)

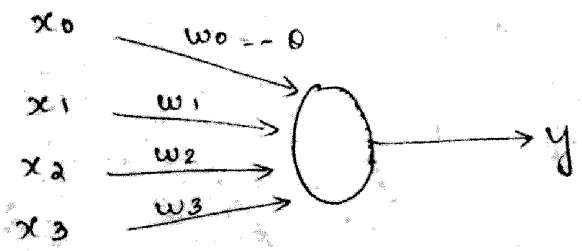
i/p	o/p
small	small
exceeds θ	large

A more accepted convention :

- $y = 1$. if $\sum_{i=0}^{n} x_i w_i \geq 0$, where $x_0 = 1$ & $w_0 = -\theta$

  Bias value
  threshold.

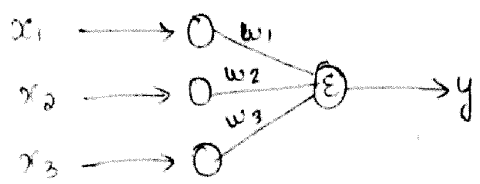
Assume, $x_0 = 1$ & $w_0 = -\theta$. then



• $y = 0$, if $\sum_{i=1}^{n} x_i w_i < 0$, where $x_0 = 1$ & $w_0 = -\theta$

Eg: PREDICTION → Based on past data

Task of predicting of liking a movie or not?



x_1 = actor A
x_2 = thriller
x_3 = director B

for example, x_1 = not A x_2 = not thriller x_3 = dir B, we can make $y = 1$ by giving more weightage to x_3

Movie bad = y = $0 = 0$

Selective viewer = y = (3) $0 = 3$

OR operation:

- Prediction based on past data
- I/p, weight & θ depend on those past data

x_1	x_2	y	
0	0	0	$\rightarrow w_0 + \sum_{i=1}^{2} x_i w_i < 0$
0	1	1	$\rightarrow w_0 + \sum_{i=1}^{2} x_i w_i \geq 0$
1	0	1	
1	1	1	

i) $w_0 + 0 \times w_1 + 0 \times w_2 < 0$

$\Rightarrow w_0 < 0$ ✓

ii) $w_0 + 0 \times w_1 + 1 \times w_2 \geq 0$

$w_0 + w_2 \geq 0 \Rightarrow w_2 \geq -w_0$

iii) $w_0 + 1 \times w_1 + 0 \times w_2 \geq 0$

$w_0 + w_1 \geq 0 \Rightarrow w_1 \geq -w_0$

iv) $w_0 + 1 \times w_1 + 1 \times w_2 \geq 0$

$w_0 + w_1 + w_2 \geq 0 \rightarrow w_1 + w_2 \geq -w_0$

- Perceptron learning algorithm

x_1	x_2	
0	0	0
0	1	1
1	0	1
1	1	1

Q. $w_0 = -1$, $w_1 = 1.1$, $w_2 = 1.1$

$$-w_0 + x_1 w_1 + x_2 w_2 < 0$$

$$-1 + x_1 1.1 + x_2 1.1 < 0$$

$$-1 + 1.1 x_1 + 1.1 x_2 < 0$$

case 1 $(x_1 = 0, x_2 = 0)$

$$-1 + 0 \times 1.1 + 0 \times 1.1 < 0$$

$$-1 < 0 \quad \checkmark \quad y = 0$$

case 2

$$-1 + 0 \times 1.1 + 1 \times 1.1 < 0$$

$$0.1 > 0 \quad \checkmark \quad y = 1$$

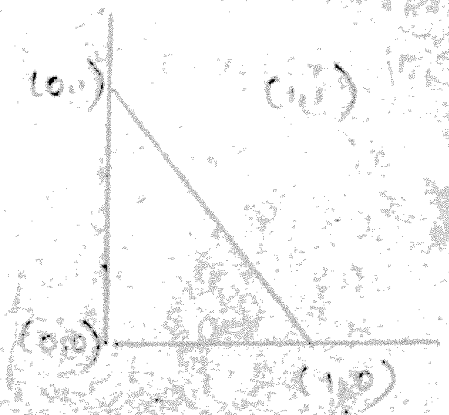
case 3

$$-1 + 1 \times 1.1 + 0 \times 1.1 < 0$$

$$0.1 > 0 \quad \checkmark \quad y = 1$$

case 4

$$-1 + 1 \times 1.1 + 1 \times 1.1 > 0 \quad \checkmark \quad y = 1$$



Q. $w_0 = -1$, $w_1 = -1$, $w_2 = -1$

$$-w_0 + x_1 w_1 + x_2 w_2 < 0$$

case 1 $(x_0 = 0, x_1 = 0)$

$$-1 + 0 \times -1 + 0 \times -1 < 0$$

$$-1 < 0 \quad (y = 0)$$

case 2

$$-1 + 0 \times -1 + 1 \times -1 < 0$$

$-2 < 0$ $(y=0)$

case 3

$-1 + 0.1 \times -1 + 0 \times -1 < 0$

$-2 < 0$ $(y=0)$

case 4

$-1 + 1 \times -1 + 1 \times -1 < 0$

$-3 < 0$ $(y=0)$

Not satisfying. (3 errors)

Q $w_0 = -1$ $w_1 = 1.5$ $w_2 = 0$

$-w_0 + w_1 x_1 + x_2 w_2 < 0$

case 1

$-1 + 0 \times 1.5 + 0 \times 0 < 0$

$-1 < 0$ $(y=0)$ ✓

case 2

$-1 + 0 \times 1.5 + 1 \times 0 < 0$

$-1 < 0$ $(y=0)$ X

case 3

$-1 + 1 \times 1.5 + 0 \times 0 < 0$

$0.5 < 0$

$0.5 > 0$ $(y=1)$ ✓

case 4

$-1 + 1 \times 1.5 + 0 \times 0 < 0$

$0.5 < 0$

$0.5 > 0$ $(y=1)$ ✓

1 error. Not satisfying

$w_1 = 0.45$ $w_2 = 0.45$

$b + x_1 w_1 + x_2 w_2 > 0$

Case 1

$-1 + 0 \times 0.45 + 0 \times 0.45 \leq 0$

$-1 < 0$ $(y=0)$ ✓

Case 2

$-1 + 0 \times 0.45 + 1 \times 0.45$

$-1 + 0.45 < 0$

$-0.55 < 0$ $(y=0)$ ✗

Case 3

$-1 + 1 \times 0.45 + 0 \times 0.45$

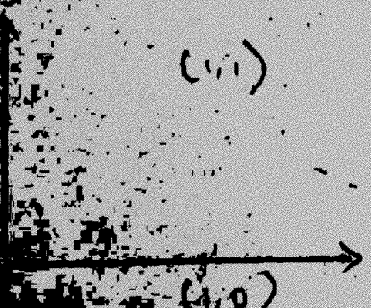
$-1 + 0.45 < 0$

$-0.55 < 0$ $(y=0)$ ✗

0.45
0.45
0.90
-1

$-1 + 1 \times 0.45 + 1 \times 0.45$

$-0.10 < 0$ $(y=0)$ ✗



Activation function (transfer function)

→ It decides whether a neuron should be activated or not by calculating weighted sum further adding bias with it

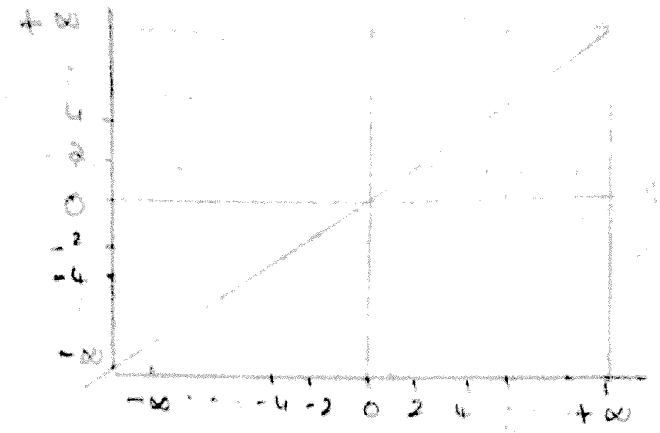
Why we use activation function?

* To determine the o/p of neural network like 1/0, YES/NO
* It maps the resulting value in b/w 0 to 1 or from -1 to 1 depending on the activation func. (Range will be dependent)
* The purpose of activation fnc is to introduce non-linearity into the o/p of the neuron
* Neural n/w have neurons that work in correspondance of weight, bias, & respective activation point;
* update the weights & bias of the neuron on the basis of the errors at the o/p. This process is known as BACK PROPAGATION
* Activation func make BACK PROPOGATION POSSIBLE.
* Activation function can be basically divided into 2

1) Linear or identity activation func.
2) Non-linear activation function

- Linear (Identity) activation function

> This func is a line or linear ∴ o/p of the func will not be confined or restricted b/w any range $(-\infty, +\infty)$

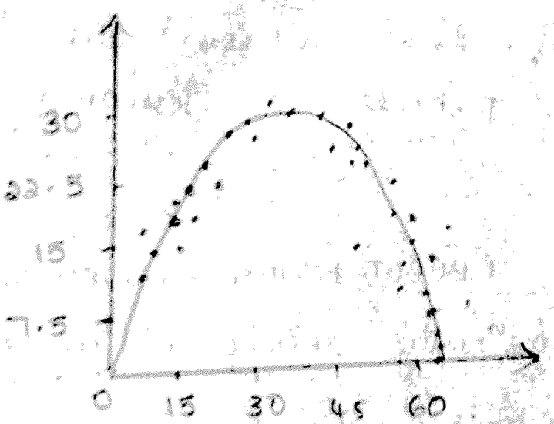


Here range is $-\infty$ to ∞.

$$f(x) = x$$

Non-linear activation functions

> These are the most used activation fun.
> It makes easy for the model to generalise or adapt with varieties of data values & differentiate b/p o/p.
> focus on solving complex task.



we don't get a straight line
we get a curve

Terminologies associated with non-linear function

① Derivative or Differential

* Changing y-axis w.r.t changing x-axis.
* also known as slop

② Monotonic function

* A fune which is ~~either~~ non either entire

decreasing or non-increasing.

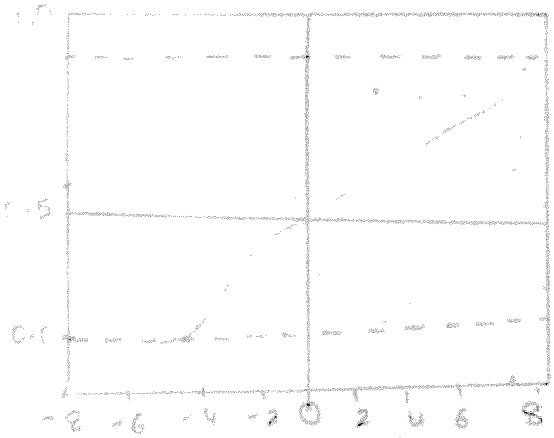
The non-linear activation function are categorised on the basis of RANGE or CURVE.

It can be categorised into

1) Sigmoid (logistic) activation func.
2) TANH or Hyperbolic Tangent activation func.
3) Relu or Rectified Linear Unit
4) Leaky Relu

1) SIGMOID Activation func.

* looks like S shaped Curve.
* Range is from 0 to 1
* function is differential and monotonic
* Used to predict probability
* func has derivative which is not monotonic
* Softmax func is a more generalised logistic activation func used for multi-class classification.



Why derivative or differentiation used ?

→ When updating the curve, to know in which direction & how much to change or to update the curve depending upon the slope

→ We use differentiation in almost every part of ML & DL

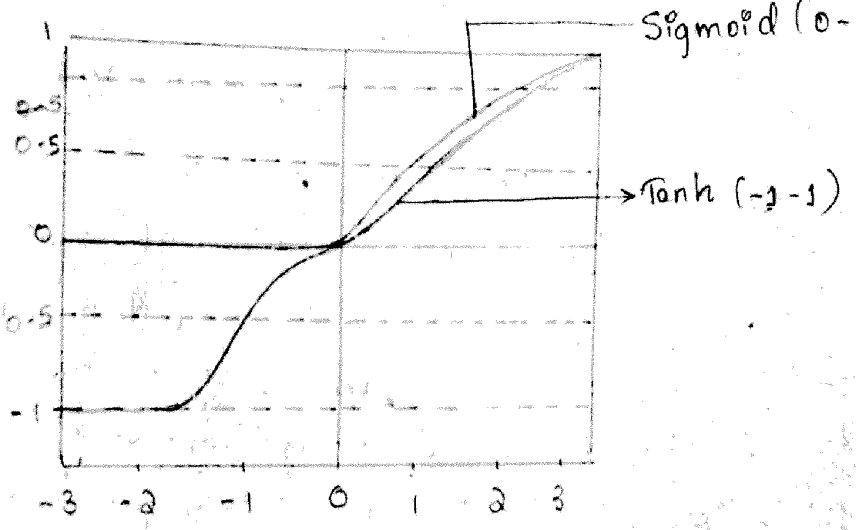
$$\sigma(z) = \frac{1}{1+e^{z}}$$

Equation $f(x) = \frac{1}{1+e^{x}}$

Derivative $f'(x) = f(x) \cdot (1 - f(x))$

2) TaNH or hyperbolic Tangent Activation func

- Similar to sigmoid, S shaped
- range from -1 to 1
- The function is differential - monotic while derivative is not monotonic.



- The Tanh func is mainly used for classification b/w two classes. (binary)

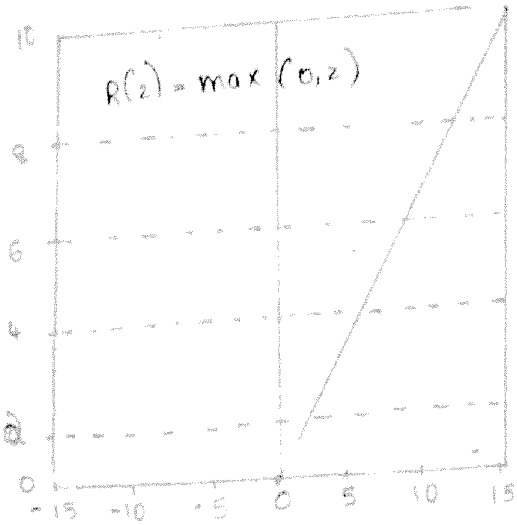
Equation = $f(x) = \frac{2}{1+e^{-2x}} - 1$

derivation = $f'(x) = 1 - f(x)^2$

- Both Tanh & Sigmoid Activation functions are used in feed-forward neural n/w.

3) ReLu

* most frequently or commonly used activation func.
* used in CNN (convolutional neural Network)
* Range is 0 to ∞.
* Both function & derivative are monotonic



* It acquires Half - Rectified (no -ve values)
* $f(z) = 0$ when $z < 0$ (negative)
* $f(z) = z$ when $z >= 0$
* Equation : $f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

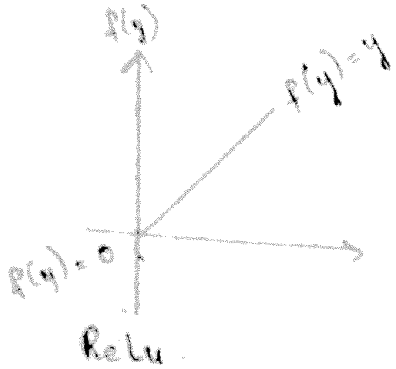
$$\text{derivation} : f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

* Issue :

> All the negative values become zero immediately in the graph which in turn affects the resulting graph by not mapping -ve values appropriately.

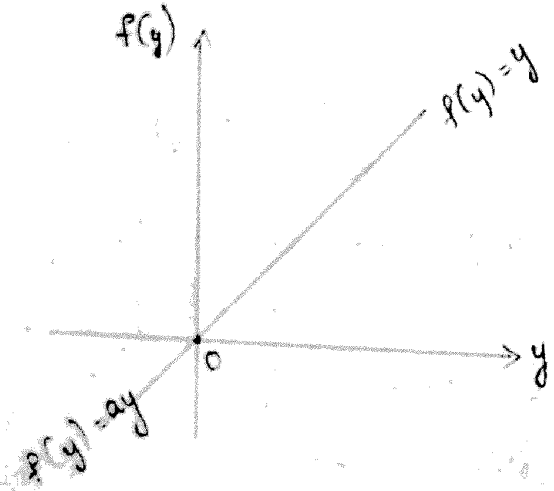
> It decreases the ability of the model to fit or train from the data properly.

- To solve dying Relu problem, Leaky Relu was introduced
- It helps to increase the range of Relu function.
- Taking the parameter 'a' = 0.01
- Range is $-\infty$ to ∞.



Relu

- Here in Relu, f(y) is from 0 to y



- Monotonic (for func. & derivative) in nature.
- Equation : $f(x) = \begin{cases} ax & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$

Derivative : $f'(x) = \begin{cases} a & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

$a = 0.01 \longrightarrow$ leaky Relu

parameter value, $a \neq 0.01 \longrightarrow$ Randomised ReLu.

↓

differential & derivative are monotonic

Single layer Perceptron

→ It is a neural network with a set of inputs.

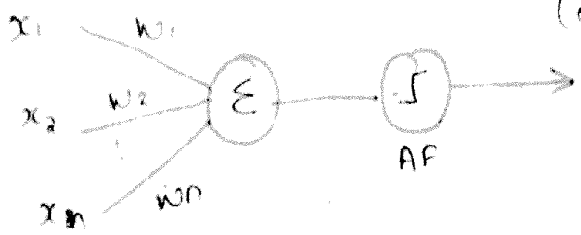
→ It consist of two layers

① Input layer

② Output layer

- produce single binary o/p.

→ Each input have weight → used to decide the o/p



$x_i w_i \geq 0$

→ Add biased value (w_0) to sum of weighted input.

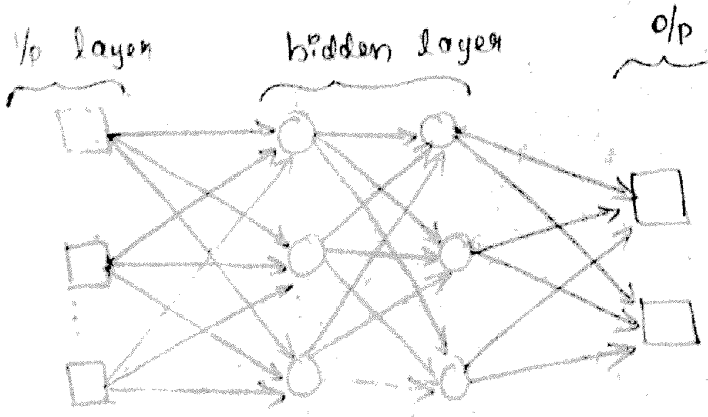
Multi Layer Perceptron (MLP)

- Most complicated structure
- It consist of multiple layer & therefore known as multilayer Perceptron.
- Multiple layers of n/w of perceptron
- Consist of 3 layers

① input layer (single)

② hidden layers (any no of)

③ Ouput layer (single)



- Used for Supervised learning
- Typical learning algo for MLP network also call

BACK PROPOGATION ALGORITHM

- I/p layer receives I/p signals to be processed, then it is given hidden layer.
- Hidden layer & o/p layer performs computations
- o/p layer gives prediction or classification o/p
- Hidden layer is the computation Engine
- Arbitary no. of hidden layers are placed in b/w layer & o/p layer, are the true computation Engine of
- Data flow in forward direction (I/p ⟶ o/p)
- The neurons in MLP are trained with the b Propogation learning algo which can solve problem which are not linearly Separable.
- The major areas of MLP are :
  1. Pattern Recognition.
  2. Classification
  3. Prediction & Approximation.
- The computations taking place at every neuron in hidden layer & o/p layer.

Hidden: $h(x) = S(b(1) + w(1))$ ⟶ $h(x)$

$h(x)$ — hidden layer; $b(1)$ — bias; $w(1)$ — weight; S — Activation func:

Output: $o(x) = G(b(2) + w(2)h(x))$ $h(x) \xrightarrow{V_2}$ o/p

$o(x)$ — output layer

- $b(1)$, $b(2)$ are bias vectors.
- $w(1)$, $w(2)$ are weight matrices.

• S & G are Activation function

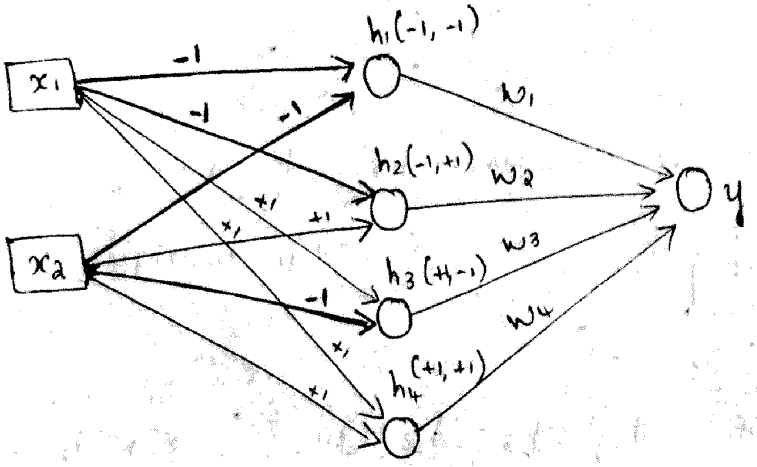
• Set of parameters to learn are

$$Set(\theta) = \{b(1), b(2), w(1), w(2)\}$$

by using back Propogation Algorithm.

• For MLP, we typically use TanH or Sigmoid activation function

→ 2 I/P 4 Perceptrons O/P layer



Black edge represents $w = -1$

blue edge indicates $w = +1$

Bias of each perceptron $w_0 = -2$.

* Each perceptron will fire only if weighted sum of I/P is greater than or equal to -2.
* Each of these perceptron in the hidden layer is connected to the perceptron in output layer by weights.
* Also these weight should be learnt.
* ~~Red~~ Black & blue edges are called layer1 weights.
* w_1, w_2, w_3, w_4 are called layer2 weights.
* Checking this n/w can be used to implement any Boolean func. (Linearly Separable ones)

> Each perceptron in the hidden layer fires

only for the specific input

(for h_1, $x_1 = -1$ & $x_2 = -1$ h_3 $x_1 = +1$ & $x_2 = -1$

h_2, $x_1 = -1$ & $x_2 = +1$ h_4 $x_1 = +1$ & $x_2 = +1$

)

XOR Operation

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

w_0 is the bias o/p of the neuron. It will fire if

$$\sum_{i=1}^{4} w_i h_i \geq w_0$$ (4 perceptron)

x_1	x_2	XOR	h_1	h_2	h_3	h_4	$\sum_{i=1}^{4} w_i h_i \geq w_0$
0	0	0	1	0	0	0	w_1
0	1	1	0	1	0	0	w_2
1	0	1	0	0	1	0	w_3
1	1	0	0	0	0	1	w_4

} weight factor

* Those results in the 4 condition to implement

$w_1 < w_0$ (1)

$w_2 \geq w_0$ (2)

$w_3 \geq w_0$ (3)

$w_4 < w_0$ (4)

* Each w_i is now responsible for one of the 4 possible inputs & can be adjusted to get the desired for that i/p.

Theorem :

Any Boolean function of n' input can be represented exactly by a network of perceptron containing one hidden layer with 2^n perceptrons & one output layer containing one perceptron



Eg: A person would like to buy a car or not with single input $x \longrightarrow$ Salary.

Sal in $\frac{K}{1000}$	y (buy or not)
80	1
20	0
65	1

Theorem :

Any Boolean function of n' input can be represented exactly by a network of perceptron containing one hidden layer with 2^n perceptrons & one output layer containing one perceptron.

n i/p → (HP) → (P)

Eg A person would like to buy a car or not with sing input x ⟶ Salary.

Sal in $\frac{K}{1000}$	y (buy or not)
80	1
20	0
65	1
15	0
30	0
~~49~~ 49	0
51	1
87	1



Left side → Person who donot buy a car

Right side → Person who buy a car

- In Perceptron model, a small change in the i/p will completely flip the o/p in different way. Flip is fro 0 to 1. This change is completely depend on weight an threshold.
- Used to handle linearly separable data or binary classification.
- This is because of the characteristic of neuron wh is itself act. ~~step~~ as a step function (Reason for flip)

↓

which increase or decrease abruptly from one constant va to another constant value.

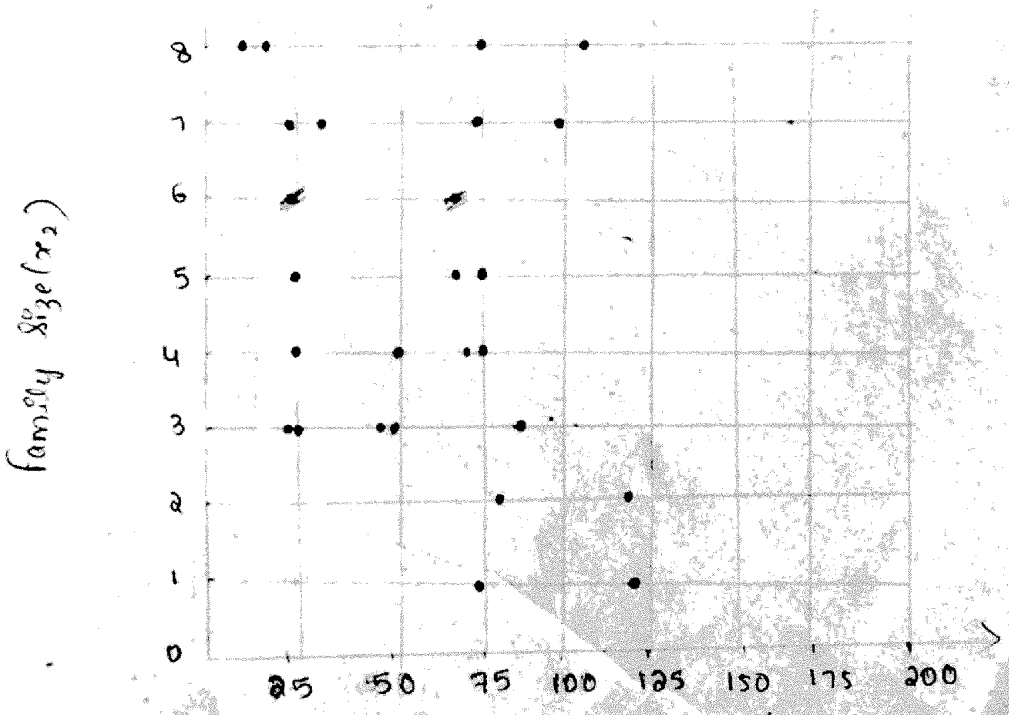
- To avoid this problem, another artificial neuron called SIGMOID NEURON
- In this model, output is smooth that perceptron m
- Here small change in i/p will cause small (minor) change in o/p
- Used to handle non-linearly separable data.
- It is like a S-like shape
- Commonly used sigmoid function is
  - logistic
  - soft max

$$y = \frac{1}{1 + e^{-(wx+b)}}$$

- y is not either 0 or 1 (between 0 & 1)

Eg. A person would like to buy a car or not based on two inputs :

Salary lpa (x_1)	Size of family (x_2)	buy or not
11	8	1
20	7	1
4	8	0
8	7	0
11	5	1



> Here number of iterations are performed to minimize error.

> A single ~~neuron~~ sigmoid neuron cannot handle non-linear data.

→ we need many Sigmoid neuron for nonlinear

↓

MLP → multiple layers of neurons

→ also called as feed forward Neural network or deep feed forward Neural network.

→ Hidden layer in MLP will have non-linear data

(energy) (Assignment) : related